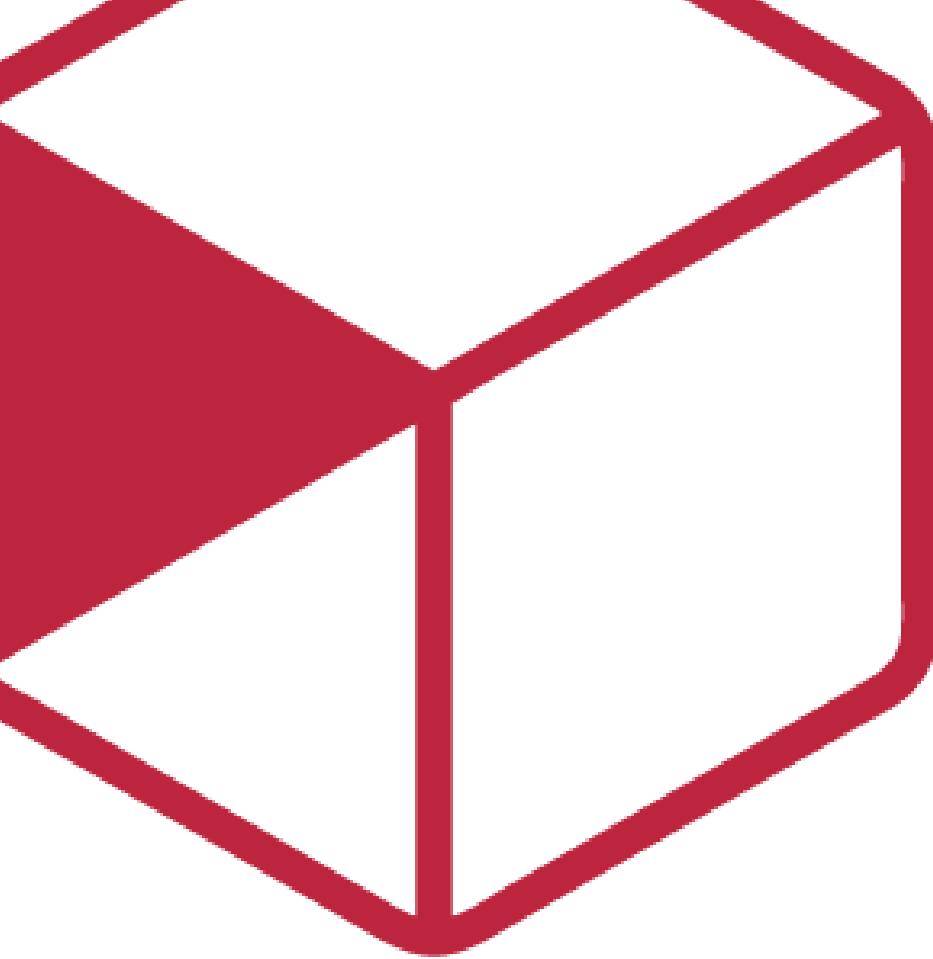




[PROPUESTA]

Proquifa.Pqf2.Notificaciones

Propuesta de solución

FORMATO	N/A
PROYECTO	R16A
REFERENCIA	N/A
VERSIÓN	1.0
FECHA	04/07/2026
AUTOR	Osmar Calderón
REVISOR	Valdemar Farina Sánchez

Contenido



1. Objetivo

Proponer la creación de `Proquifa.Pqf2.Notificaciones` como **único canal de envío de correo** de las soluciones de Proquifa, presentar su diseño de fondo y obtener la decisión que permita arrancar.

2. Contexto actual

Hoy el correo lo envía ProquifaDotNet 2.0 por su cuenta. Habla directamente con Brevo —el proveedor de envío transaccional, antes llamado SendInBlue—, arma el contenido con **diez plantillas XSLT** que él mismo transforma, y deja el registro de cada envío en la tabla `CorreoEnviado`, alrededor de la cual hay **más de diez tablas satélite** y código apuntándole.

No es el único que escribe ahí. La solución de Finanzas guarda sus correos en **la misma base y la misma tabla**, aunque nunca consulta su estado: el único que lee esos datos es la vista que decide cómo se ve una cotización. Cualquier cambio sobre esa tabla afecta a dos soluciones, no a una.

Y hay más soluciones esperando turno. Finanzas y Timbrado necesitan enviar correo, y la regla del proyecto es explícita: **ninguna solución integra al proveedor por su cuenta**, todo pasa por un canal único. Existe además un requisito previo para un servicio de envío separado —R16A-N0-FU-001—, cuyo alcance se absorbe aquí, y **seis requisitos detenidos** esperando que este contrato exista.

Nada de esto es un sistema roto. Lleva años funcionando y manda correos todos los días. Pero antes de proponer un reemplazo se leyó el código que los envía, porque **lo que se hereda sin mirar se hereda completo**, defectos incluidos. Estas observaciones salen de ahí, no de suponerlo:

Qué se encontró	Por qué importa
La fecha de envío se escribe en el `INSERT` , no cuando el correo sale	El dato dice "enviado" antes de que se haya intentado enviar
El camino encolado responde antes de que exista un resultado	Quien llama no puede saber si el correo salió
Existe una ventana entre el <code>Commit()</code> y la publicación en la cola	Si el proceso muere ahí: registro guardado, cotización marcada <i>Enviada</i> , ningún mensaje en la cola . Nadie se entera
El camino síncrono sí obtiene el resultado y lo descarta — el método que envía es <code>void</code>	La información existe y se tira
El adjunto se pasa como ruta a un archivo local , que se borra en la línea siguiente	Una referencia puede desaparecer antes de que el correo se procese

Dos métodos hermanos envían con remitentes distintos — uno como "Ryndem Studio", el otro como "PROQUIFA"	No hay una fuente única del remitente
---	---------------------------------------

Y cuánto correo es. Medido sobre la tabla actual, de enero a agosto de 2026:

Por día hábil	**128 correos**, que son **150 envíos** contando destinatarios
Pico	219 correos · 255 envíos , y no se ha movido en siete meses
Al mes	~2,800 correos · ~3,300 envíos
Patrón	Solo días hábiles: 156 días con actividad de 216 naturales

Dos precisiones sobre esa cifra. **Es un piso, no un techo:** esos correos son de las dos soluciones que hoy comparten la tabla, y este servicio existe para centralizar también a las que todavía no envían. Y **lo que se cobra son envíos, no correos:** un correo va en promedio a 1.17 destinatarios, así que la columna que dimensiona es la segunda.

Lo que el dato deja claro es que **el volumen es modesto y estable**. Nada aquí obliga a infraestructura especial, y el pico —1.65 veces el promedio— no esconde ráfagas.

3. Propuestas para la solución de correo

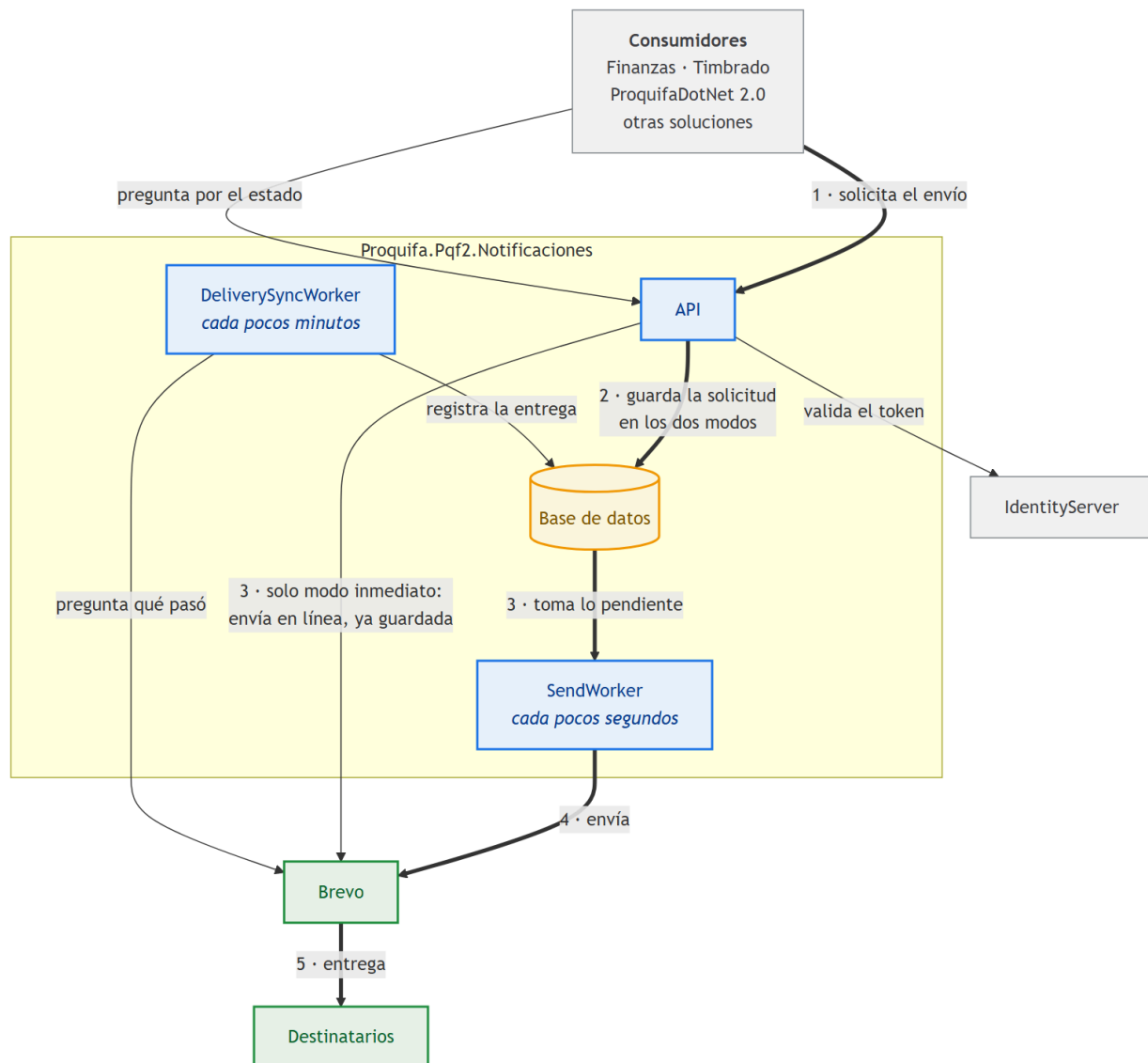
3.1 La solución en una frase

Un servicio propio —API, worker y base de datos— que **centraliza el envío de correo del ecosistema contra Brevo**, y que registra el estado de cada envío en dos ejes independientes: si logramos entregarlo a Brevo, y qué pasó después con cada destinatario.

3.2 Panorama general

El diagrama muestra las piezas y todo lo que las rodea. Se lee de arriba hacia abajo, siguiendo el camino de un correo desde que un consumidor lo solicita hasta que llega al destinatario. La línea gruesa numerada del 1 al 5 es ese camino; las delgadas son caminos reales pero secundarios.





Código Mermaid del diagrama:

```

---
config:
  flowchart:
    nodeSpacing: 30
    rankSpacing: 50
---
flowchart TB
    CONS["<b>Consumidores</b><br>Finanzas · Timbrado<br>ProquifaDotNet 2.0<br>otras soluciones"]
    IDS["IdentityServer"]

    subgraph NTF["Proquifa.Pqf2.Notificaciones"]
        API["API"]
        DB["Base de datos"]
    end

    CONS -- "1 · solicita el envío" --> API
    API -- "valida el token" --> IDS
    API -- "pregunta por el estado" --> DSW["DeliverySyncWorker<br>cada pocos minutos"]
    DSW -- "registra la entrega" --> DB
    DB -- "3 · toma lo pendiente" --> SW["SendWorker<br>cada pocos segundos"]
    SW -- "4 · envía" --> Brevo["Brevo"]
    Brevo -- "5 · entrega" --> Destinatarios["Destinatarios"]
    DSW -- "pregunta qué pasó" --> Brevo

```



```

SW["SendWorker<br/><i>cada pocos segundos</i>"]
DSW["DeliverySyncWorker<br/><i>cada pocos minutos</i>"]
end

BREVO["Brevo"]
DEST["Destinatarios"]

CONS ==>|"1 · solicita el envío"| API
API ==>|"2 · guarda la solicitud<br/>en los dos modos"| DB
DB ==>|"3 · toma lo pendiente"| SW
SW ==>|"4 · envía"| BREVO
BREVO ==>|"5 · entrega"| DEST

CONS -->|"pregunta por el estado"| API
API -->|"valida el token"| IDS
API -->|"3 · solo modo inmediato:<br/>envía en línea, ya guardada"| BREVO
DSW -->|"pregunta qué pasó"| BREVO
DSW -->|"registra la entrega"| DB

classDef externo fill:#f1f3f4,stroke:#80868b,color:#3c4043
classDef propio fill:#e8f0fe,stroke:#1a73e8,stroke-width:2px,color:#0b3d91
classDef datos fill:#fef7e0,stroke:#f29900,stroke-width:2px,color:#7f5700
classDef proveedor fill:#e6f4ea,stroke:#1e8e3e,stroke-width:2px,color:#0d652d
class CONS,IDS externo
class API,SW,DSW propio
class DB datos
class BREVO,DEST proveedor

```

Pieza	Qué hace
Consumidores	Cualquier solución que necesite enviar un correo. Ninguna se conecta a Brevo por su cuenta. ProquifaDotNet 2.0 es uno de ellos
IdentityServer	Ya existe y opera. Emite el token con el que se identifica cada consumidor; nosotros solo lo validamos
API	Recibe la solicitud, la valida, la guarda y responde. También contesta las consultas de estado
Base de datos	Propia. Guarda la solicitud, los destinatarios, los adjuntos y la bitácora de cada intento
SendWorker	Cada pocos segundos revisa la base, envía lo pendiente y reintenta lo que falla
DeliverySyncWorker	Cada pocos minutos pregunta a Brevo qué pasó con lo ya enviado, y lo registra

Brevo	Único punto de contacto con el exterior. Entrega los correos y reporta qué pasó con cada uno
-------	---

Los dos workers viven en **un solo proceso**, no en dos ejecutables.

3.3 La decisión de fondo: guardar antes de intentar

La solicitud queda guardada antes de cualquier intento de envío, en los dos modos. Eso cierra la ventana entre el `Commit()` y la cola que hoy puede dejar una cotización marcada como enviada sin que exista ningún mensaje, y convierte a la base de datos en **la fuente de verdad del despacho**: el worker no espera a que le avisen, revisa la tabla.

Es el patrón **outbox**, con una variante que conviene declarar: en la formulación canónica hay una tabla de outbox separada y el relevo publica mensajes a un broker. Aquí **la tabla de solicitudes hace de outbox y el relevo ejecuta el envío**, en vez de publicar. La propiedad esencial se conserva —un solo commit local y un relevo asíncrono con reintentos—, pero el nombre merece la precisión.

3.4 Los dos ejes de estado

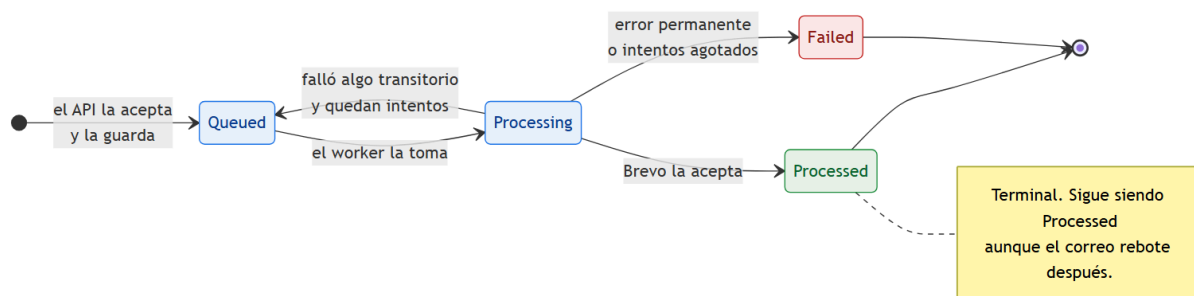
Un correo no tiene *un* estado, tiene **dos**, y confundirlos es el error que este diseño evita. La palabra "enviado" es ambigua: puede significar "*Brevo lo aceptó*" o "*le llegó al cliente*". Son hechos distintos, separados por horas, que fallan por causas distintas.

	Eje 1 — la solicitud	**Eje 2 — la entrega**
Responde	¿Hicimos nuestro trabajo?	¿Qué pasó con el correo allá afuera?
Quién lo determina	Nosotros	Brevo nos lo reporta
Granularidad	Una por petición	Una por destinatario
Forma	Máquina de estados — excluyentes y ordenados	Marcas con fecha y hora — ni excluyentes ni ordenadas
Se puede quedar vacío	No. Siempre termina en Processed O Failed	Sí. Puede no llegar nunca ninguna marca

3.4.1 Eje 1 — la solicitud

Es una máquina de estados de verdad: los estados son excluyentes y siempre se llega a un final. **No hay cancelación**. El ciclo `Processing` → `Queued` es el reintento; se recorre hasta cinco veces, espaciando cada vez más el siguiente intento.





Código Mermaid del diagrama:

```

---
config:
  look: classic
---
stateDiagram-v2
  direction LR
  [*] --> Queued: el API la acepta<br/>y la guarda
  Queued --> Processing: el worker la toma
  Processing --> Processed: Brevo la acepta
  Processing --> Queued: falló algo transitorio<br/>y quedan intentos
  Processing --> Failed: error permanente<br/>o intentos agotados
  Processed --> [*]
  Failed --> [*]

  note right of Processed
    Terminal. Sigue siendo Processed
    aunque el correo rebote después.
  end note

  classDef ok fill:#e6f4ea,stroke:#1e8e3e,color:#0d652d
  classDef mal fill:#fce8e6,stroke:#c5221f,color:#8c1d18
  classDef curso fill:#e8f0fe,stroke:#1a73e8,color:#0b3d91
  class Processed ok
  class Failed mal
  class Queued,Processing curso
  
```

Estado	Significa
Queued	Aceptada y guardada, esperando su turno. Ya pasó todas las validaciones
Processing	Un worker la tomó y está intentando enviarla. Existe para que dos workers no tomen el mismo renglón
Processed	Brevo la aceptó y nos dio su identificador de mensaje



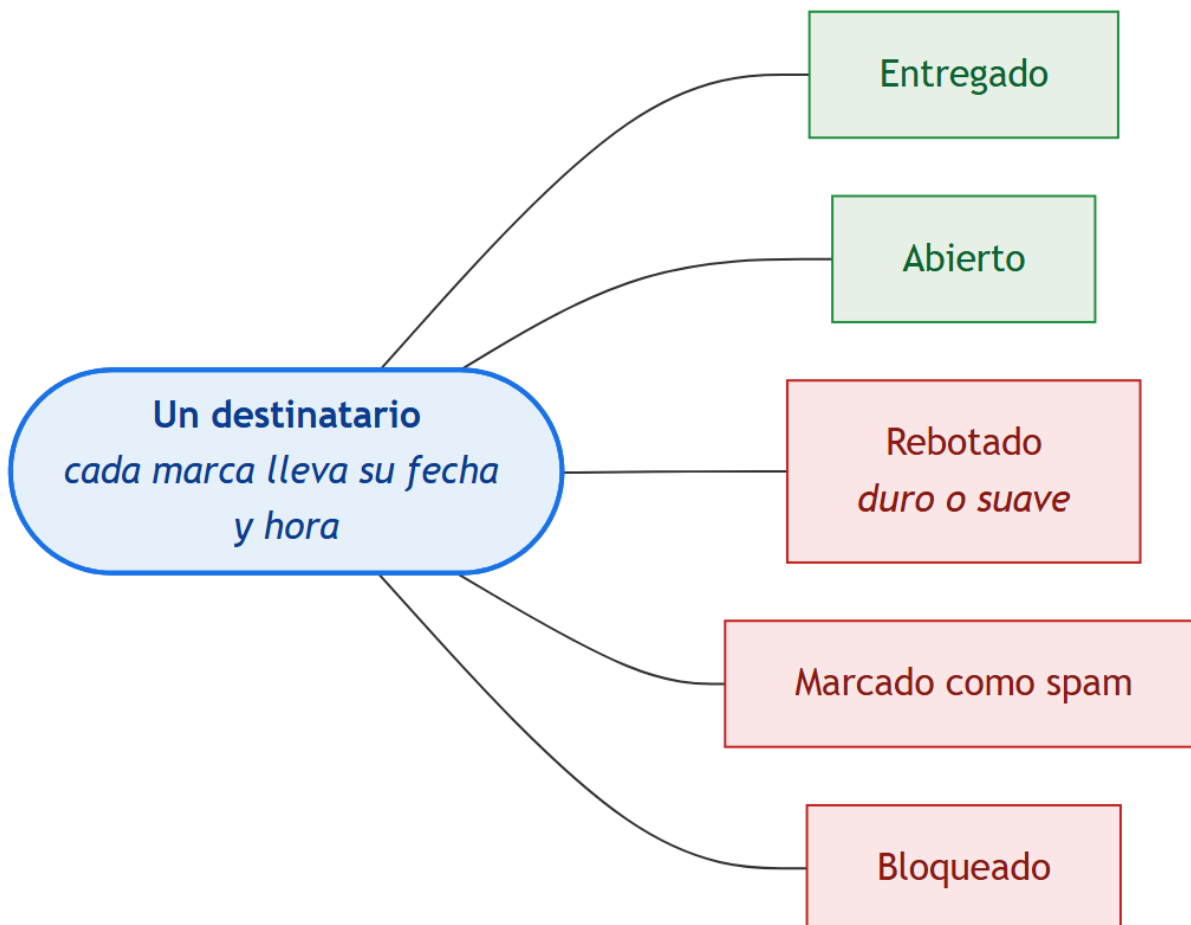
Failed	Error permanente, o se agotaron los cinco intentos
--------	--

`Processed` es terminal aunque el correo rebote después. No es un descuido: el rebote pertenece al eje 2 y no revierte el eje 1, porque nuestro trabajo —entregárselo a Brevo— sí se cumplió.

3.4.2 Eje 2 — la entrega, por destinatario

Esto no es una máquina de estados. Son marcas independientes, cada una con su fecha y hora, que se ponen sobre un destinatario. En el diagrama las líneas **no llevan punta de flecha**, precisamente porque no son transiciones.

Y solo se siguen para los destinatarios directos. Las copias se registran —hay que saber a quién se copió— pero **no se les da seguimiento**: si el correo a una copia rebota, este servicio no lo va a reportar. Es una simplificación deliberada, y tiene un costo: la copia suele ser una dirección interna, así que un buzón propio roto no se detecta desde aquí.



Código Mermaid del diagrama:

```

---
config:
  flowchart:
    nodeSpacing: 25
    rankSpacing: 70
---
flowchart LR
  R(["<b>Un destinatario</b><br/><i>cada marca lleva su fecha y hora</i>"])
  R --- D["Entregado"]
  R --- O["Abierto"]
  R --- B["Rebotado<br/><i>duro o suave</i>"]
  R --- S["Marcado como spam"]
  R --- BL["Bloqueado"]

  classDef centro fill:#e8f0fe,stroke:#1a73e8,stroke-width:2px,color:#0b3d91
  classDef bueno fill:#e6f4ea,stroke:#1e8e3e,color:#0d652d
  classDef malo fill:#fce8e6,stroke:#c5221f,color:#8c1d18
  class R centro
  class D,O bueno
  class B,S,BL malo

```

Tres propiedades hacen imposible modelarlo como un enumerado: **no están ordenadas** —se puede marcar como spam después de haber abierto—, **no son excluyentes** —entregado y abierto conviven— y **pueden no llegar nunca**, o llegar desordenadas o repetidas.

De ahí que la regla para escribirlas sea "*si está vacío, ponlo*": recibir el mismo evento dos veces o fuera de orden no rompe nada.

Marca	Significa
Entregado	El servidor del destinatario lo aceptó
Abierto	No es prueba de lectura. Depende de un pixel de rastreo, que falla con imágenes bloqueadas, vista previa o precarga del cliente de correo
Rebotado	<i>Duro</i> : la dirección no existe, es permanente. <i>Suave</i> : temporal — buzón lleno, servidor caído
Marcado como spam	El destinatario lo reportó. Daña la reputación del dominio
Bloqueado	Brevo se negó a entregar — normalmente porque la dirección ya rebotó antes o reportó spam, y él mantiene su propia lista

El eje 2 **empieza donde el eje 1 termina**, y cambia la granularidad: una solicitud Processed puede tener tres destinatarios —uno entregado y abierto, uno rebotado, y uno del que nunca se sepa nada.

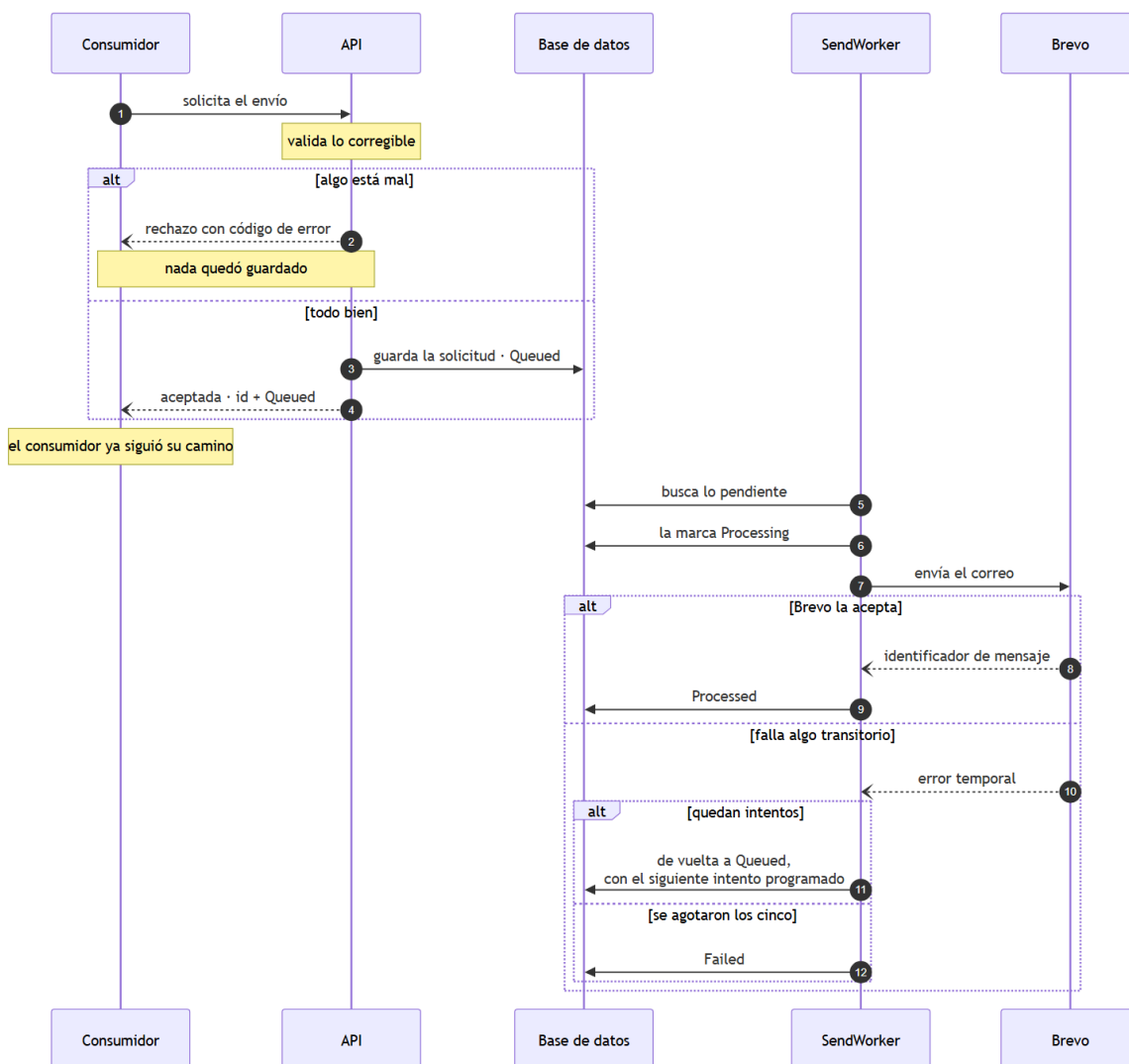


3.5 El flujo de envío

Hay **dos modos**, en dos endpoints distintos. Hacen lo mismo; lo único que cambia es en qué punto del ciclo responden. Las dos validan primero y **guardan antes de intentar**.

3.5.1 Modo encolado — el camino por omisión

El API responde en cuanto la solicitud queda guardada. El envío pasa después, en el worker, y el consumidor ya no está esperando.



Código Mermaid del diagrama:

```

sequenceDiagram
    autonumber
    participant C as Consumidor
    participant A as API
    participant D as Base de datos
    participant W as SendWorker
    participant B as Brevo
    
```

```

C->>A: solicita el envío
Note over A: valida lo corregible
alt algo está mal
    A-->C: rechazo con código de error
    Note over A,C: nada quedó guardado
else todo bien
    A->>D: guarda la solicitud · Queued
    A-->C: aceptada · id + Queued
end

Note over C: el consumidor ya siguió su camino

W->>D: busca lo pendiente
W->>D: la marca Processing
W->>B: envía el correo

alt Brevo la acepta
    B-->W: identificador de mensaje
    W->>D: Processed
else falla algo transitorio
    B-->W: error temporal
    alt quedan intentos
        W->>D: de vuelta a Queued,<br/>con el siguiente intento programado
    else se agotaron los cinco
        W->>D: Failed
    end
end
end

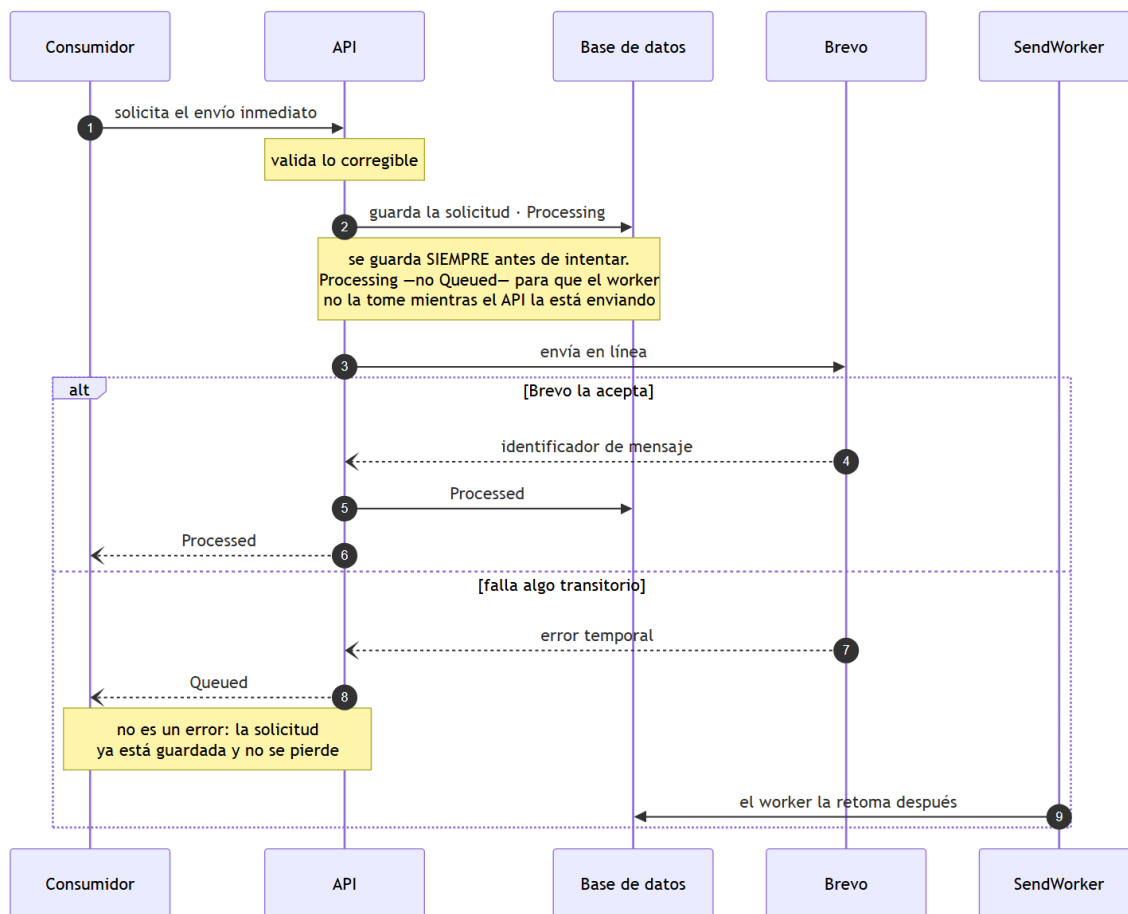
```

El rechazo por validación no guarda nada	Si la dirección está mal, la plantilla no existe o falta un parámetro, se rechaza mientras el consumidor todavía escucha. No queda un renglón a medias
El reintento no es un ciclo infinito	Cinco intentos, cada uno más espaciado. Al agotarlos la solicitud queda en Failed, y esos renglones son la cola de fallidos



3.5.2 Modo inmediato

El API intenta el envío en línea y responde con el resultado. Sirve para quien necesita saber ya si el correo salió.



Código Mermaid del diagrama:

```

sequenceDiagram
    autonumber
    participant C as Consumidor
    participant A as API
    participant D as Base de datos
    participant B as Brevo
    participant W as SendWorker

    C->>A: solicita el envío inmediato
    Note over A: valida lo corregible
    A->>D: guarda la solicitud · Processing
    Note over A,D: se guarda SIEMPRE antes de intentar.<br/>Processing –no Queued– para que el  
worker<br/>no la tome mientras el API la está enviando
    A->>B: envía en línea

    alt Brevo la acepta
        B-->>A: identificador de mensaje
        A->>D: Processed
    else [falla algo transitorio]
        B-->>A: error temporal
        A->>W: Queued
        W->>A: el worker la retoma después
    end
    A->>C: no es un error: la solicitud  
ya está guardada y no se pierde
  
```

```

A--)C: Processed
else falla algo transitorio
B--)A: error temporal
A--)C: Queued
Note over C,A: no es un error: la solicitud<br/>ya está guardada y no se pierde
W->>D: el worker la retoma después
end

```

El punto importante está en la rama de falla. Cuando Brevo no responde, el API **no devuelve un error: devuelve `Queued`**. No es una respuesta a medias, es la correcta — la solicitud ya está guardada y el worker la va a recoger.

La razón de fondo: el consumidor no reintenta la llamada una vez que recibió respuesta. Si en ese momento le devolviéramos un error, el correo se perdería.

3.6 El worker

Un solo proyecto, un solo ejecutable, y dentro de él **dos servicios en segundo plano** con ciclos e intervalos independientes.

3.6.1 El despacho

Cómo se evita que dos workers tomen el mismo renglón. El estado *en proceso* lo reserva, y la marca de tiempo es lo que hace la diferencia: sin ella no se podría distinguir "*alguien lo está trabajando*" de "*alguien murió trabajándolo*". Con la hora, pasado un umbral el renglón se considera colgado y vuelve a estar disponible. Es también lo que permite repartir el trabajo entre varias instancias sin coordinación externa.

La escalera de reintentos es una sola: un intento = una llamada HTTP = un renglón en la bitácora.

Intentos	5
Espera	Base de 1 minuto, duplicándose: 1, 2, 4, 8, 16 — unos 31 minutos de cobertura
Variación aleatoria	Sí, sumada a cada espera
Al agotarse	Failed con el código del último error y se emite la alerta . El aviso al consumidor se cubre en el apartado siguiente

Dos precisiones que conviene declarar, porque las dos se cuestionan:

Se desactiva el reintento en proceso de la librería de resiliencia para la llamada a Brevo.

Si se dejara activo, se anidaría con la escalera del outbox y las llamadas se **multiplicarían** —tres por cinco son quince—, además de que la bitácora mentiría: quedaría un renglón por ciclo aunque hubieran salido tres llamadas. Todo el espaciado vive en un solo lugar.

La variación aleatoria no es un adorno. Si Brevo cae, cientos de correos fallan en el mismo instante; sin variación, todos reintentarían justo cuando se está recuperando, y lo volverían a tumbar.

3.6.2 La actualización del estado de entrega

El segundo servicio le pregunta a Brevo por los envíos **cuyo estado de entrega todavía puede cambiar**, apoyándose en un índice propio para no recorrer todo lo enviado desde el inicio de los tiempos.

Aplica cada evento con la regla *"si está vacío, ponlo"*, de modo que **es indiferente al orden y a los duplicados**. No toca el eje 1: un rebote no revierte un Processed.

Dos consultas periódicas distintas, que conviene no confundir. *La del despacho consulta nuestra propia tabla y mueve el eje 1 — es la que sustituye al broker. La del estado de entrega le pregunta a Brevo y mueve el eje 2. Son independientes, tienen cadencias distintas y ninguna depende de la otra.*

Se consideró que Brevo nos avisara, y se optó por preguntarle. La alternativa era exponer una dirección pública a la que el proveedor mandara cada evento en cuanto ocurre: menos espera y menos consultas. Se descartó para el arranque por tres razones.

No sustituiría a esto, se sumaría	Los avisos **se pierden** : basta un despliegue nuestro de cinco minutos para que el proveedor no encuentre a nadie, reintente un rato y se rinda. Y no se pierde un correo, se pierden **todos los que se entregaron en esa ventana** . El dato sigue existiendo del lado del proveedor, así que **preguntar lo recupera** — pero solo si hay alguien que vaya a preguntar. Sin eso, esos correos se quedan sin estado de entrega para siempre
Exige una dirección expuesta a internet	Con lo que eso implica de infraestructura y de seguridad. No es una decisión nuestra: es una petición a quien opera la red
El proveedor no firma sus avisos	Verificado en su documentación: ofrece un token en cada llamada y recomienda filtrar por sus direcciones de origen . Funciona, pero es más débil —el token viaja en cada llamada, así que se filtra por donde se filtran los secretos— y el filtro por direcciones tiene mantenimiento : el proveedor las cambia, y si se queda viejo los avisos se caen en silencio



La pieza queda prevista, no descartada. Quien aplica un evento de entrega es **un solo componente**, y tanto la consulta periódica como un futuro receptor de avisos son nada más adaptadores de entrada que le delegan. Agregar esa vía después **no cambia el diseño**: es una clase más y la dirección pública que hoy no tenemos.

3.6.3 Qué pasa si el worker está caído

Nada se despacha, pero nada se pierde. Las solicitudes se acumulan en la tabla y salen cuando el worker vuelva. El API sigue aceptando peticiones con normalidad, y **el modo inmediato sigue funcionando**, porque no pasa por el worker.

El hueco real no es la pérdida, es la ceguera: hoy nadie se enteraría. El endpoint de salud expone la antigüedad de la solicitud pendiente más vieja —que es la señal exacta—, pero no hay monitoreo externo que lo consulte. Está registrado como hueco conocido.

3.7 El contrato de entrada

El detalle completo vive en Contrato-API.md. Lo esencial:

Método	Ruta	Qué hace
POST	/api/v1/email	Acepta el correo y lo encola. Responde de inmediato
POST	/api/v1/email/immediate	Acepta el correo e intenta enviarlo en línea antes de responder
GET	/api/v1/email/{id}	Estado de una solicitud
GET	/api/v1/email?externalReference=...	Estado por la referencia del consumidor

Idempotencia. El consumidor manda una **referencia externa propia**, con restricción de unicidad. Una sola pieza resuelve tres cosas: no duplicar, correlacionar de punta a punta, y recuperarse de un tiempo de espera agotado. Si la referencia se repite, se devuelve la solicitud original con 200 — **nunca se envía un segundo correo**.

Errores. Catálogo con prefijo NTF-, respondidos como application/problem+json con código y correlación. El criterio transversal: **lo que un humano puede corregir se rechaza mientras el consumidor todavía escucha** —dirección mal formada, plantilla inexistente, parámetros faltantes, adjunto excedido—; lo transitorio se reintenta.

3.8 Modelo de datos

La estructura completa vive en DDL-Notificaciones.sql. **Es borrador pendiente de revisión.**

Tabla	Rol
EmailRequest	La solicitud y el eje 1
EmailRecipient	Un renglón por destinatario y el eje 2

EmailAttempt	Un renglón por intento — interno, no se expone en el contrato
EmailAttachment	Metadatos del adjunto y la referencia al archivo original. El contenido binario se libera al confirmar el envío
EmailTemplate	Clave lógica → plantilla de Brevo, más los parámetros que exige. La misma clave sirve en todos los ambientes y regiones
ProviderConfiguration	Credenciales y remitente por región
AppSetting	Valores de configuración del aplicativo

4. Propuestas para el manejo de plantillas

4.1 El problema

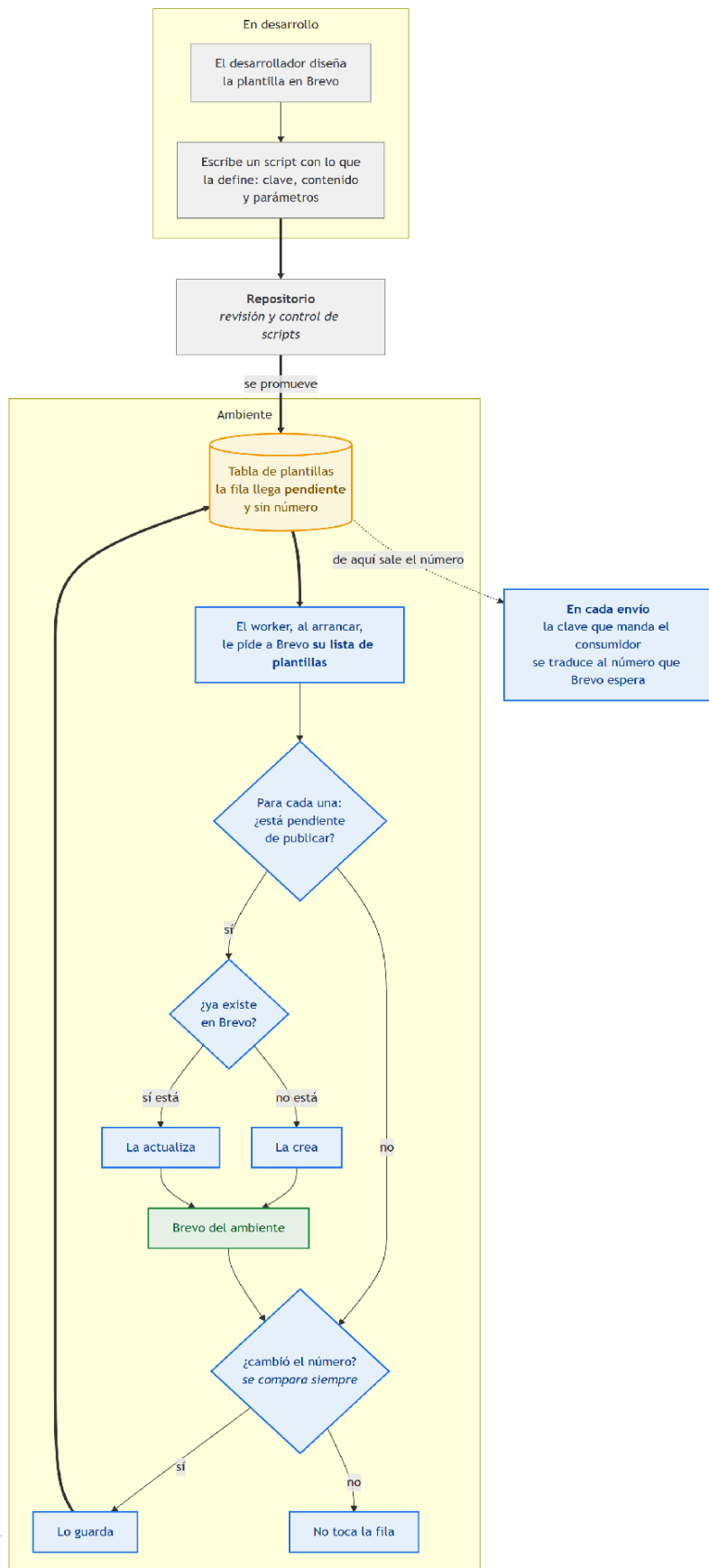
El consumidor manda una **clave lógica** —FACTURA, NOTA_CREDITO—. Brevo, en cambio, exige un **número**, y ese número **lo asigna cada cuenta**: la misma plantilla tiene un identificador distinto en cada ambiente.

Alguien tiene que traducir, y esa traducción vive en una tabla nuestra. La regla de fondo es una sola: **el número nunca viaja entre ambientes** — se resuelve en cada uno, contra el Brevo que le toca.

4.2 El recorrido de una plantilla

Se diseña **una vez** en desarrollo, viaja en un script versionado que pasa por revisión, y en cada ambiente el worker la publica y anota el número que ese Brevo le devuelve. **En ningún punto alguien copia un número a mano** — que es donde estaba el error posible, y era un error mudo: un número mal transcrito que apuntara a otra plantilla existente enviaría el correo equivocado sin fallar.





```

config:
  flowchart:
    nodeSpacing: 40
    rankSpacing: 55
    subGraphTitleMargin:
      top: 10
      bottom: 10
---
flowchart TB
  subgraph DES["En desarrollo"]
    DEV["El desarrollador diseña<br/>la plantilla en Brevo"]
    SC["Escribe un script con lo que<br/>la define: clave, contenido<br/>y parámetros"]
    DEV --> SC
  end

  REPO["<b>Repositorio</b><br/><i>revisión y control de scripts</i>"]

  subgraph AMB["Ambiente"]
    TAB["Tabla de plantillas<br/>la fila llega <b>pendiente</b><br/>y sin número"]
    LISTA["El worker, al arrancar,<br/>le pide a Brevo <b>su lista de plantillas</b>"]
    PEND["Para cada una:<br/>¿está pendiente<br/>de publicar?"]
    EX["¿ya existe<br/>en Brevo?"]
    CREA["La crea"]
    ACT["La actualiza"]
    CAMBIO["¿cambió el número?<br/><i>se compara siempre</i>"]
    GUARDA["Lo guarda"]
    IGUAL["No toca la fila"]
    BREVO["Brevo del ambiente"]
    LISTA --> PEND
    PEND -->|"sí"| EX
    PEND -->|"no"| CAMBIO
    EX -->|"no está"| CREA
    EX -->|"sí está"| ACT
    CREA --> BREVO
    ACT --> BREVO
    BREVO --> CAMBIO
    CAMBIO -->|"sí"| GUARDA
    CAMBIO -->|"no"| IGUAL
  end

  ENV["<b>En cada envío</b><br/>la clave que manda el consumidor<br/>se traduce al número que Brevo espera"]

  SC ==> REPO
  REPO ==>|"se promueve"| TAB
  TAB ==> LISTA
  GUARDA ==> TAB
  TAB -->|"de aquí sale el número"| ENV

  classDef externo fill:#f1f3f4,stroke:#80868b,color:#3c4043
  classDef propio fill:#e8f0fe,stroke:#1a73e8,stroke-width:2px,color:#0b3d91
  classDef datos fill:#fef7e0,stroke:#f29900,stroke-width:2px,color:#7f5700

```



```
classDef proveedor fill:#e6f4ea,stroke:#1e8e3e,stroke-width:2px,color:#0d652d
class DEV,SC,REPO externo
class LISTA,PEND,EX,CREA,ACT,CAMBIO,GUARDA,IGUAL,ENV propio
class TAB datos
class BREVO proveedor
```

Con esto, **copiar una plantilla de un ambiente a otro deja de ser trabajo manual**, y el contenido vuelve al control de versiones —con diferencias, revisión y vuelta atrás—, que era lo único que se perdió al llevar las plantillas al proveedor.

4.3 Las reglas que esto impone

Tres, y conviene comunicarlas porque la interfaz de Brevo no las hace cumplir:

El nombre en Brevo es la clave, exactamente	El emparejamiento se hace contra él. Una plantilla creada desde la interfaz debe llamarse `NOTA_CREDITO`, no Nota de crédito
Renombrar no es cosmético: es un cambio de contrato	La clave es lo que los consumidores mandan en cada petición. En la práctica, una plantilla en uso tiene nombre inmutable . Si alguien renombra igual, el worker lo alerta al arrancar y al usarla
En QA y producción nadie edita plantillas en la interfaz	La siguiente publicación las sobrescribe. En desarrollo sí: ahí la interfaz es el banco de trabajo

4.4 Si una misma cuenta de Brevo sirviera a varios ambientes

Todo lo anterior asume que **cada ambiente tiene su propia cuenta**. Si no fuera así —si desarrollo, QA y producción compartieran una— aparece un problema serio: **al publicar desde desarrollo se sobrescribiría la plantilla que usa producción**, porque sería literalmente la misma.

Para ese caso hay salida, y conviene tenerla escrita antes de necesitarla: **plantillas por ambiente dentro de la misma cuenta**. Cada ambiente publica sobre una plantilla con nombre propio —QA_FACTURA, PROD_FACTURA— mientras **el consumidor sigue mandando la misma clave de siempre**, FACTURA. La tabla guarda a qué nombre de Brevo apunta cada ambiente, y el contrato no se entera.

Qué resuelve	Publicar desde un ambiente deja de poder pisar a otro, que es el riesgo grave
Qué cuesta	Una columna más en la tabla y tantas plantillas en Brevo como ambientes por cada correo



Qué NO resuelve	Reputación, cuota y lista de supresión siguen compartidas : un error en QA puede dañar la entregabilidad de producción. Eso solo lo arregla separar cuentas
-----------------	--

Es la opción de repliegue, no la preferida. Se documenta para que la decisión sobre las cuentas se tome sabiendo que existe alternativa, y no bajo la impresión de que separar cuentas es la única forma de que esto funcione.

5. Propuestas para la actualización de la tabla de correos de Proquifa

La tabla de correos del sistema actual tiene que reflejar lo que pasó con cada envío. Este apartado describe **qué hay que mantener al día, cómo se obtiene el dato, y las tres formas de construirlo** entre las que hay que elegir.

5.1 Qué hay que mantener al día, y por qué

La tabla de correos del sistema actual **no tiene ningún dato que diga que un envío sigue en curso**, así que la vista que muestra el estado de una cotización lo adivina con un temporizador de 40 minutos: durante ese rato reporta *recibido* sin saber nada —ocultando un rebote que ya ocurrió— y después reporta *error* aunque el correo haya llegado perfectamente, de modo que **los mismos datos dan resultados distintos según la hora en que se consulten**. Lo que hay que mantener al día es exactamente lo que hoy falta: **el estado del envío, la fecha de rebote y el identificador de nuestra solicitud**, tres columnas nuevas que este servicio sí produce y con las que la vista deja de adivinar. El dato llega **colapsado a la fila única** que esa tabla usa —la marca más temprana entre los destinatarios directos, ignorando las copias, porque un vendedor abriendo la suya marcaría la cotización como confirmada por el cliente—, **sin migración de datos**, y el temporizador se conserva solo como puente mientras dure la convivencia. Un detalle que condiciona todo lo que sigue: **esa tabla la comparten ProquifaDotNet 2.0 y la solución de Finanzas**, así que lo que se construya mantiene los renglones de las dos o deja una tabla donde unos son confiables y otros no.

5.2 Opción 1 · Un worker del lado del consumidor

Un servicio instalado en su lado, con su propio temporizador. Lee su tabla, nos pregunta por los correos que todavía pueden cambiar, colapsa el resultado y lo escribe.

Pregunta correo por correo, con su propia clave — la referencia que él mismo generó al pedir el envío. Es la única que tiene garantizada: si nuestra respuesta se perdiera por la red no tendría nuestro identificador, y ese es justo el correo del que menos sabe. **Y solo pregunta por lo que todavía puede cambiar**: uno entregado, rebotado, o pasada la ventana en que el estado se mueve, no se vuelve a consultar. Ese criterio es lo que evita que el trabajo crezca sin fin — **lo acota lo que está en vuelo, no el histórico**.

A favor	Todo pasa por el contrato ya diseñado: nuestro esquema y el suyo quedan libres de tocarse sin avisar. Y funciona **aunque las dos bases vivan en servidores distintos**, que es el caso que no sabemos
En contra	Es un artefacto nuevo que alguien tiene que construir, instalar y vigilar. Hace una llamada por cada correo no resuelto en cada pasada. Y el criterio para colapsar —que es la parte delicada— lo implementa quien lo construya, leyendo un documento

Por qué pregunta él, en vez de que nosotros le avisemos. *La alternativa era publicar cada cambio de estado en un broker. Se descartó porque publicar recrea el problema que este diseño existe para arreglar: guardar el estado y publicar el mensaje son dos sistemas sin transacción común, y hacerlo bien exigiría otra tabla intermedia con su propio relevo — más maquinaria que una consulta. Además el requisito de latencia aquí es de minutos, no de segundos: quien consume el estado es una vista que decide cómo se ve una cotización. Y si algún día hiciera falta reaccionar más rápido, agregar la publicación después no obliga a rehacer nada.*

>

Por qué uno por uno y no un listado de cambios. *Se evaluó exponer todo lo que cambió desde un punto, para recorrerlo con un marcador. Ahorra llamadas, pero su corrección dependía de dos detalles nada evidentes y ambos fallan en silencio: uno hace que se salten registros para siempre, el otro que se recorra una y otra vez un hueco que crece. Poner esa sutileza a implementar en otro equipo no compensa el ahorro — el error no se nota hasta que alguien pregunta por qué una cotización lleva semanas sin estado.*

5.3 Opción 2 · Un trabajo programado de base de datos contra una vista nuestra

Publicamos una vista con el estado **ya colapsado**. **El trabajo programado corre del lado de ProquifaDotNet 2.0**, en su propia instancia y con el agente de base de datos que su equipo ya opera: lee nuestra vista y actualiza su tabla en una sola sentencia.

Igual que la opción 1, **el trabajo es de su lado** — lo que cambia es que aquí no hay nada que construir, instalar ni vigilar por separado.



A favor	**No hay artefacto nuevo ni disparador nuevo**: el agente de base de datos ya existe y ya se opera. Trabaja **por conjunto** , no correo por correo. Y **la parte delicada queda de nuestro lado** , donde está el conocimiento — su lado se vuelve una copia sin criterio
En contra	La vista se convierte en un contrato que no podemos cambiar a la ligera. Y exige que las dos bases se alcancen entre sí , lo que depende de dónde quede alojada cada una — dato que todavía no tenemos

No contradice el principio de no escribir en base ajena. *Ese principio dice que nosotros no tocamos su base. Aquí es al revés: ellos leen de la nuestra, a través de algo que publicamos a propósito.*

5.4 Opción 3 · Que la escribamos nosotros

Nuestro propio worker(Notificaciones) escribe directamente en su tabla.

A favor	**Cero trabajo de su lado.** Ni proceso, ni trabajo programado, ni alta en ningún lado
En contra	Su esquema pasa a ser nuestro problema para siempre. Con dos consumidores son dos esquemas ajenos que mantener, y el siguiente que llegue esperará lo mismo. Es exactamente lo que el diseño descartó al decidir no escribir en base ajena



5.5 Comparación

	Dónde corre	Qué hay que construir	Trabajo por pasada	Dónde vive el criterio delicado	Qué exige
Opción 1	Del lado del consumidor	Un servicio nuevo, con su instalación y su vigilancia	Una llamada por cada correo no resuelto	Del lado del consumidor	Darse de alta como consumidor autenticado
Opción 2	Del lado del consumidor, en el agente de base de datos que ya opera	Una vista de nuestro lado y una sentencia del suyo	Una sentencia	Del lado nuestro	Que las dos bases se alcancen
Opción 3	Del lado nuestro	Nada, para ellos	Ninguna	Del lado nuestro	Renunciar a no tocar base ajena

Hacia dónde nos inclinamos. Por la **opción 1**, porque es la única que funciona sin saber dónde queda alojada cada base, y porque mantiene todo detrás del contrato.

Pero falta un dato para decidirlo bien: si las dos bases comparten instancia, la opción 2 pierde su única objeción seria y pasa a ser la más barata con diferencia — sin artefacto que construir, sin disparador que vigilar, y con la parte delicada de nuestro lado en vez de delegada.

Conviene averiguarlo antes de comprometer la opción 1.

Glosario

Término	Significado
Outbox	Patrón que resuelve la imposibilidad de comprometer atómicamente una escritura en base de datos y una publicación en un broker: se escribe todo en una sola transacción local y un proceso aparte hace después el trabajo externo
Claim check	Patrón en que un mensaje voluminoso se guarda una vez y lo que circula es solo su identificador



CQRS	Separación explícita entre operaciones que modifican y operaciones que consultan
DB-first	El script SQL es la fuente de verdad del esquema; las clases se generan a partir de la base
Scaffold	Generación de las clases de acceso a datos a partir de una base existente
Rebote duro	La dirección no existe. Es permanente: insistir solo daña la reputación
Rebote suave	Fallo temporal — buzón lleno, servidor caído. No suprime
Lista de supresión	Direcciones a las que ya no se envía porque rebotaron o reportaron spam. La mantiene Brevo , no este servicio
Reputación del remitente	Calificación que los proveedores de correo asignan a un dominio. Determina si el correo llega a la bandeja o a spam. Es un recurso compartido por todo lo que envía desde ese dominio
SPF, DKIM, DMARC	Registros DNS con que el dominio autoriza a un tercero a enviar en su nombre. Sin ellos el correo se marca como sospechoso
Espaciado creciente con variación aleatoria	Cada reintento espera más que el anterior, más un margen aleatorio para que muchos fallos simultáneos no reintenten todos en el mismo instante
Cola de fallidos	Donde terminan los mensajes que agotaron sus reintentos. Aquí no es una cola aparte: son los renglones en estado <code>Failed</code>
Idempotencia	Que repetir la misma operación produzca el mismo resultado, sin efectos duplicados
`ApiCallerMail`	El componente que la solución de Finanzas usará para llamar a este servicio. Está definido en su requisito base, con supuestos anteriores a este diseño



Proyección	Copia del estado en las tablas del consumidor, derivada de nuestra base. No es fuente de verdad
Doble escribir	Intentar comprometer atómicamente una escritura en base de datos y una publicación en un broker. No existe transacción común entre los dos, así que siempre hay una ventana en que uno ocurrió y el otro no

Control de versiones

Versión	Fecha	Autor	Tipo de Cambio	Descripción del cambio	Aprobó
1.0	29/07/2026	Osmar Calderón	Creación	Creación del documento.	

